



CSI247: Data Structures

Lecture 16 – Stack implementation

Department of Computer Science

B. Gopolang (247-275)

Previous lesson

- Queue:
 - Data stored uses FIFO technique
 - Entry: at the end
 - Removals: in front
- Queue implementation options
 - Using an array
 - Need to worry about capacity
 - Using a LinkedList
 - Permit dynamic additions and removals without worrying about capacity

Today ...

Implementation of a Stack ADT

LECTURE OBJECTIVES

- By the end of this lecture you must be able to:
 - Implement a Stack class using:
 - An array
 - Linked List

1. Stack Review

- A linear data structure
 - 2 main operations: **push()** and **pop()**
- **Additions** and **deletions** are restricted to one end
 - (AKA “top”)
 - Stack uses LIFO principle!
 - Adding new element: **push** operation
 - Removing element: **pop** operation
 - Checking top element: **peek** operation
- Many stack application areas, e.g.:
 - Recursion
 - Undo mechanisms in text editor
 - Web browsers – “Back” button
 - Evaluation of expression

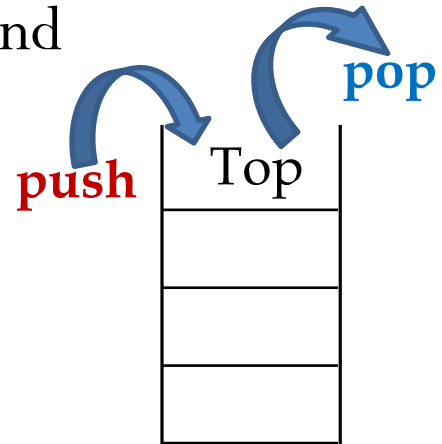
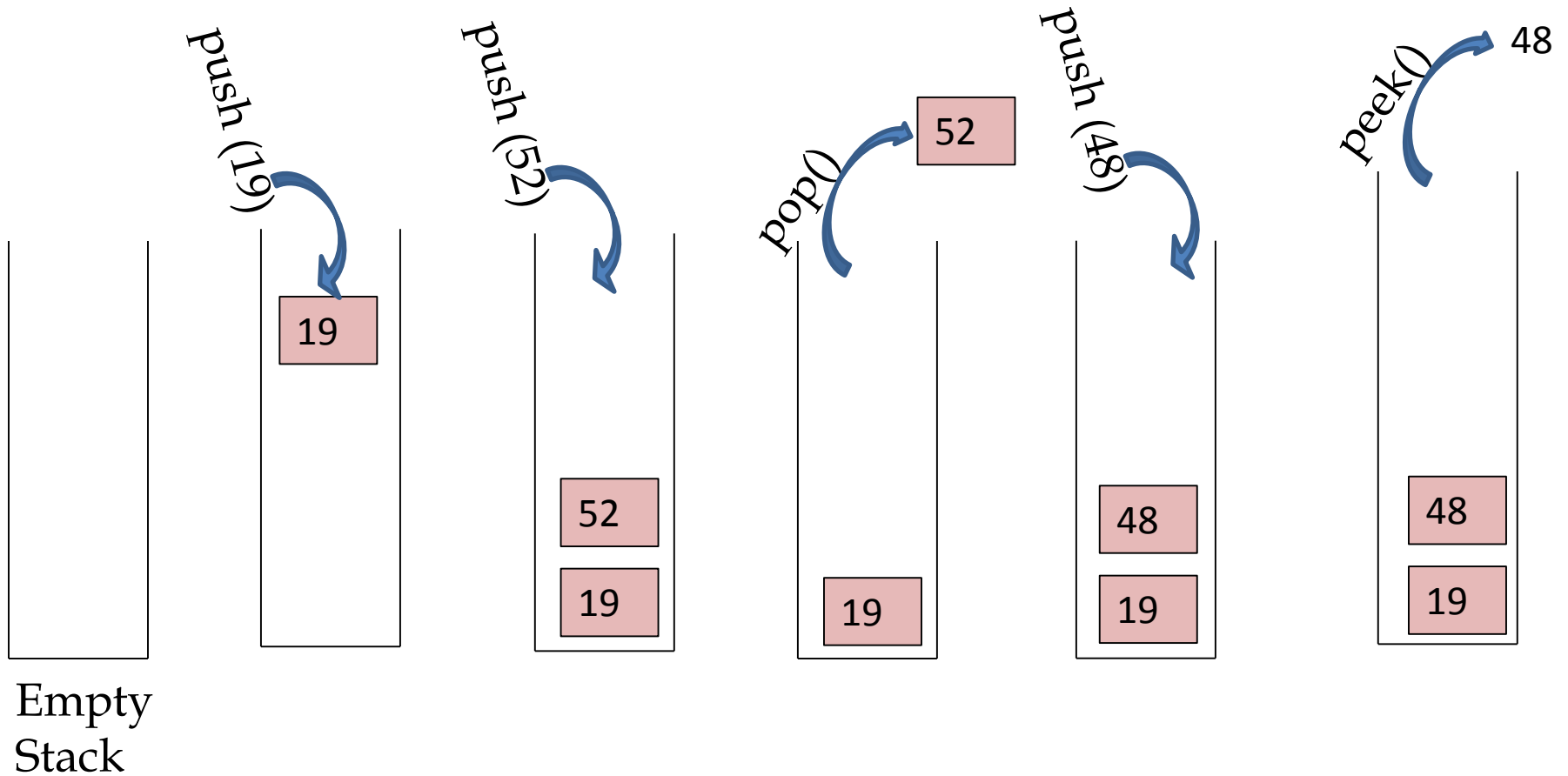


Illustration of Stack methods



Exercise 1

- a) Perform the following operations on an empty Stack:
push(14), push(39), push(26), pop(), push(28), push(17), pop(), pop(), push(87) and pop()
- b) What is the sequence of values that are popped out?
- c) How many elements are left in the stack?
- d) Which item will be popped next if there is any left?

Note: Consecutive execution of `pop()`

returns stack elements in reverse order of `push()`

2. Stack Implementation

- 2 implementation options
- a) Using an array
 - Simple and efficient implementation approach
 - All stack operations (push, pop, etc.) must be implemented using an array
 - Uses a fixed array size (determines stack capacity)
 - What if capacity is too small?
 - We can reallocate, but is an expensive approach
 - What if capacity is too large?
 - Space is wasted
 - Careful: **StackOverflow!!!** (Rem: **ArrayOutOfBounds?**)

b) Using a Linked List

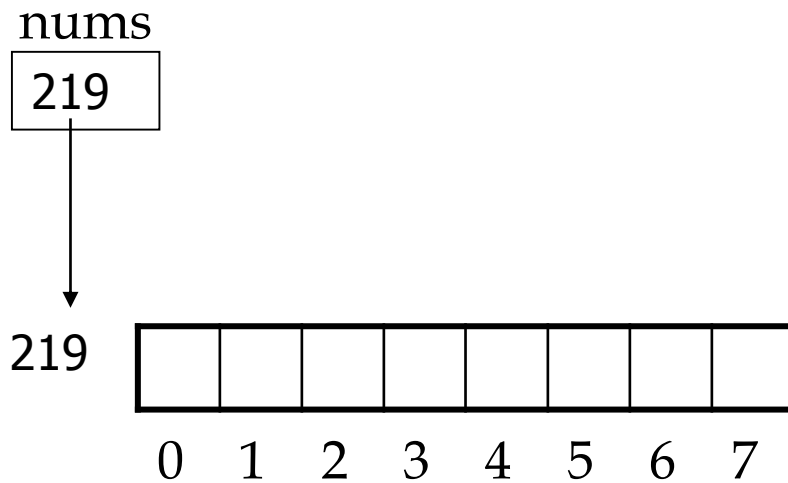
- No space limitation
- Extra space allocated for each element added (Node)
- Stack will grow & shrink dynamically

c) Using Generics

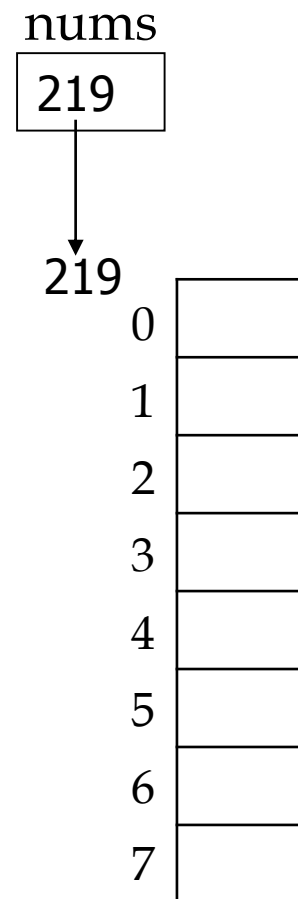
- Will accept a type parameter at object creation
- Can work with both
 - LinkedList implementation
 - array implementation

a) Array implementation of a Stack

- Rem array representation:

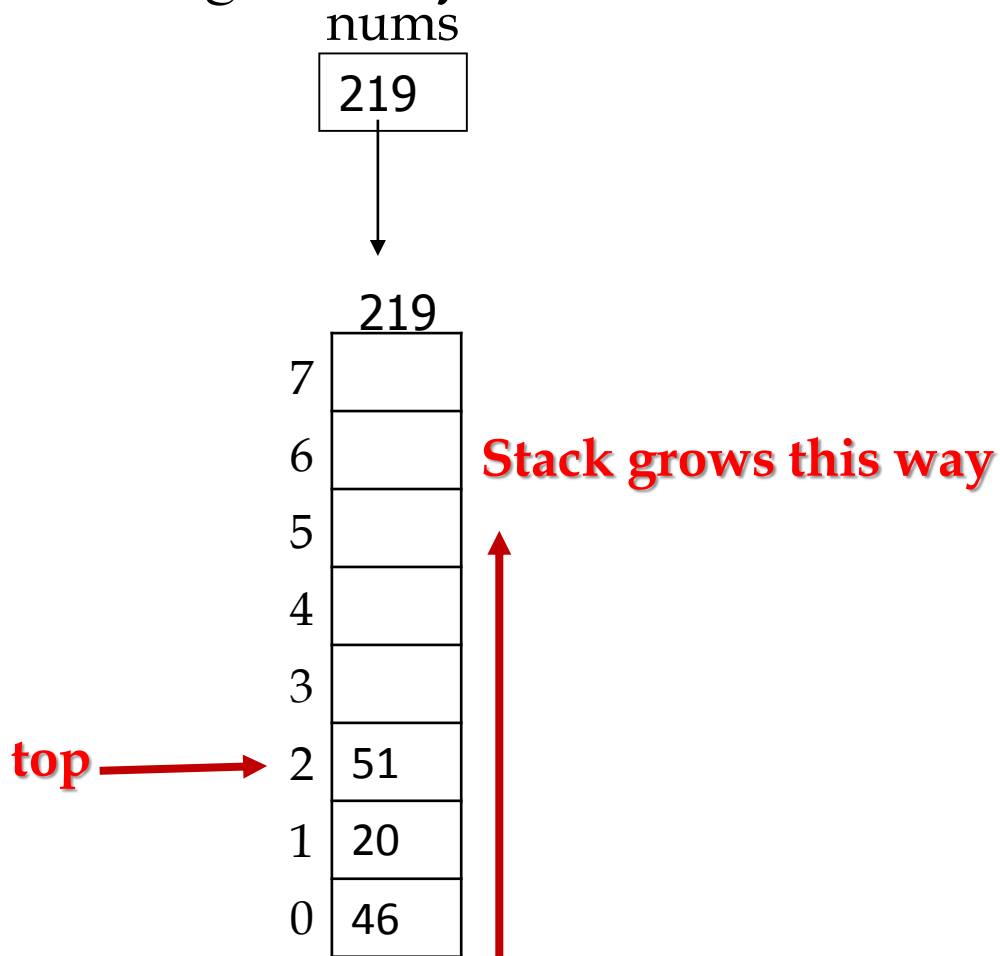


or



a) Array implementation of a Stack

- Using an array in a Stack ADT :



- Rem (about Stacks):
 - **Push, pop & peek** happen at the top
 - Hence, need to keep track of **top**
 - An index value because we are using an array
- Top depends on status of the stack as follows:

Stack status	Top
Empty	-1
Has 1 element	0
Full (has N elements)	$N - 1$
Overflow	N

- **Stack operations (methods) to implement**
 - **push()**: inserts an item at top of the stack
 - **pop()**: removes and returns item at top of the stack
 - **peek()**: returns item at top of the stack
 - Item is NOT removed!!!
 - **size()**: returns number of elements in the stack
 - **isEmpty()**: determines if stack has elements
 - Deletion NOT ALLOWED!
 - **isFull()**: determines if stack is full
 - Insertion NOT ALLOWED!

Initial look at our Stack implementation

Let us call our class Stack1, and it will store Integer values

```
public class Stack1 {  
    // instance variables  
    private Integer[] stack;    // an array to hold elements  
    private int top;           // keeps track of index of top element  
    // Rest of methods - to be added as we build the class  
}
```

Writing Stack methods ...

i. Constructor

- It must
 - Create an array of a specified size
 - Initialise top to -1 // -1 used to indicate an empty stack

```
public Stack1(int n){  
    stack = new Integer[n]; // creates array to hold n elements  
    top = -1;                // initialize index of element at the top  
}
```

ii. isEmpty()

- For checking if Stack has elements or not
- Need to check this before removing items & peeking
- Rem: **top** variable? // keeps track of top element

```
public boolean isEmpty() {  
    return (top == -1) ;  
}
```

- Note:
 - statement inside () evaluates to true when top is -1, false otherwise

iii. size()

- For determining number of elements currently in the stack
- We can use **top**: it stores index of top element
- BUT +1: // array indexed from 0 while top is initially -1

```
public int size(){  
    return top + 1;  
}
```

iv. isFull()

- For checking if Stack is full
- Do this before adding(push) items into the stack
 - Rem: **top** and **stack** instance variable?

```
public boolean isFull(){  
    return (top == stack.length - 1);  
}
```

- Note: statement inside () evaluates to **true** when they are equal, **false** otherwise

v. push()

- For inserting an element into the Stack
 - Rem: need to check if Stack is not yet full!

```
public boolean push(Integer num){  
    if (isFull()) {  
        System.out.println("Stack Overflow! Can not insert...");  
        return false;  
    }  
    else {  
        stack[++top] = num; // increase top by 1 & add at new top  
        return true;  
    }  
}
```

vi. peek()

- For checking & returning top element
- Rem: need to check if stack is not empty!

```
public Integer peek() {  
    if (!isEmpty()) {  
        return stack[top];  
    }  
    else  
        return null;  
}
```

vii. pop()

- Removes top element
 - Rem: check if stack is not empty!

```
public Integer pop() {  
    if (!isEmpty()) {  
        return stack[top--]; //return top element & decrease top by 1  
    }  
    else {  
        System.out.println("Stack Underflow! Nothing to remove");  
        return null;  
    }  
}
```

viii. search()

```
public int search(Integer key){  
    int ind = -1;  
    if (!isEmpty()) {  
        for (int i = top; i > -1; i--) { // start from top  
            if (key.equals(stack[i]) // Found!  
                ind = i; break;  
            }  
        }  
        return ind;  
    }  
    // Note: returns index of elements when found
```

Incomplete sample tester class

```
Stack1 st1 = new Stack1(5);  
// add items  
st1.push(27);  
st1.push(38);  
st1.push(92);  
st1.push(46);  
St1.push(83);  
// st1.push(90); ?
```

```
System.out.println("Top item " +  
st1.peak());  
System.out.println("Item :" + st1.pop());  
System.out.println("Item :" + st1.pop());  
System.out.println("Size" + st1.size());  
System.out.println("Found : " +  
st1.search(6));  
System.out.println("Top element : " +  
st1.peak());
```

Exercise 2

- a) Type all code segments provided into a Java file
- b) Compile it.
 - Fix any errors, if any
- c) Implement a tester class to test all the methods.

b) Implementing Stack using a Linked List

- Rem:
 - LinkedList: a list of Nodes, joined by pointers!
 - So, we still need Node inner class
 - List grows/shrinks dynamically
 - No size restriction
 - No need for isFull()
 - Elements (Nodes) added at the front (top)
 - Need a **head (top)**

● Stack operations (methods) to implement

- **push()**: inserts an item at top of the stack
- **pop()**: removes and returns item at top of the stack
- **peek()**: returns item at top of the stack
 - Item is NOT removed!!!
- **size()**: returns number of elements in the stack
- **isEmpty()**: determines if stack has elements
 - Cannot delete if empty!
- ~~**isFull()**: determines if stack is full~~
 - No size restriction!

Initial look at our implementation

Let us call our class Stack2, and it will store String values

```
public class Stack2 {  
    private class Node {  
        String data; // node's data part  
        Node next; // a pointer to next node in the Stack  
        Node(String data) { // Node Constructor  
            this.data = data;  
            this.next = null;  
        }  
    }  
}
```

Writing Stack methods

Instance variables

public Node top; // to keep track of Top node

i. Constructor

- It will be used to create an empty Stack2 object

```
public Stack2(){  
    top = null;  
}
```

ii. isEmpty()

- Checks if Stack has Nodes or not

```
public boolean isEmpty() {  
    return top == null;  
}
```

iii. size()

- Returns number of Nodes in the Stack

```
public int size() {  
    // Can you think of possible implementations?  
}
```

iv. push()

- Insert Node in the Stack

```
public void push(String data){  
    Node newNode = new Node (data);  
    if (isEmpty()){  
        top = newNode; // 1st node  
    }
```

```
else {  
    Node oldTop = top;  
    top = newNode; // new top  
    newNode.next = oldTop;  
}  
}
```

*// This is adding a node in front
of the LinkedList!*

iv. peek()

- Returns top Node (Does not delete it!)

```
public String peek() {  
    if (isEmpty() ) { // Nothing to peek()  
        System.out.println("Stack is empty");  
        return null;  
    }  
    else  
        return top.data; // return value of top Node  
}
```

v. pop()

- Removes and returns top Node

```
public String pop() {  
    if(isEmpty()) {  
        System.out.println("Stack is  
empty");  
        return null;  
    }  
}
```

else{

```
String data = top.data;  
top = top.next; // new top  
return data;
```

}

}

*// Removes node from front of the
LinkedList!*

vi. search()

- This method must return checking if a given element is in the stack
- Take-home work!
- Note
 - Method must accept an element to look for
 - It must return the index of the element if found, otherwise -1 (not found)

Incomplete sample tester class

// Create Stack object

```
Stack2 stk = new Stack2();
```

// add elements

```
st2.push("CSI247");
```

```
st2.push("CSI243");
```

```
st2.push("CSI141");
```

```
st2.push("CSI142");
```

```
st2.push("CSI262");
```

// Remove nodes

```
st2.pop(); // removes CSI262
```

```
st2.pop(); // removes CSI142
```

// Check top element

```
System.out.println("Top: " + st2.  
peek());
```

// Output?

Exercise 2

- a) Type all code segments provided into a Java file and compile it. Fix any errors, if any
- b) Implement a tester class to test all the methods.
- c) Write a Java program that reads a String from the keyboard then display the reversed String using push and pop Stack operations.
- d) Create a generic LinkedList-based stack and call it GenStack2.
- e) Test it by creating stack objects of Integer, String and Double
- f) Implement another tester class to test methods implemented in d) above, using Student class objects

Advantages vs disadvantages

Advantages

- Easy to implement
 - Operations done on one side (the top)
- Top element is quickly accessible

Disadvantages

- No random access (Rem LIFO?)
- Possibilities of Stack overflow if not implemented properly
- Limited Capacity if implemented using arrays
- Limited functionality, E.g. cannot modify middle element unless we pop all those before it first

More exercises

- Implement a class that accommodates other data types -

➔ Generics!

- a) Implement a generic array-based Stack and call it GenStack1.
Class GenStack1 must have 2 instance variables:
 - An ArrayList
 - Rem: You will need to use ArrayList methods to manipulate the ArrayList, not [] to access data elements
 - An int variable to keep track of the top element
- a) Test it by creating and manipulating a Stack of Integer, a Stack of String and a Stack of Double

6. Summary

- Stack
- Examples of Stack application areas
- Different implementation strategies
 - Refers to structure used to store data
 - An array
 - ArrayList
 - LinkedList
- Generic implementation!
 - Using arrays or ArrayList
 - Using LinkedList

Next lesson ...

Revision!!

Q & A



Thank you